Razvijanje Linux drajvera i aplikacije za FlashAttention akcelerator za Whisper Base

Student	Veljko Veselinović, Nikola Pavlica
Broj indeksa	EE103/2021, EE240/2021
Mentor	MSc Nebojša Pilipović
Datum	22.09.2026.

1. Uvod

Razvijanje drajvera i softvera u okviru predmeta Embedded Operativni Sistemi predstavlja naredni korak u razvijanju embedded platforme čiji je cilj da ubrzava rad Whisper Base modela. Ono nastavlja od prošlog dela projekta gde smo uspešno razvili ispravan RTL model za ubrzavanje enkoder slojeva Whisper Base modela.

Cilj razvoja drajvera jeste da imamo način kojim uspešno koristimo razvijen model kako bi ubrzali algoritam u Linux softverskom okruženju koji se dalje može primeniti kao zaseban uređaj ili kao umreženi uredjaj za aplikacije koje zahtevaju usluge pretvaranja zvuka u tekst.

Takvo softversko okruženje će obuhvatati razvoj Linux kernel drajvera koji će naš razvijeni RTL akcelerator predstaviti kao uređaj koje aplikacije mogu da koriste kako bi pokrenuli enkoder slojeve Whisper Base modela. Nakon toga, napravljena je grana (eng. “fork”) popularne aplikacije za pokretanje Whisper modela pod imenom whisper.cpp koja će podržavati naš hardver u cilju da se ubrza pokretanje Whisper Base modela.

2. Specifikacije razvojnog sistema i arhitektura modela

Zybo Z7-10 ploča na kojoj smo razvili RTL model sadrži XC7Z010-1CLG400C System-on-a-Chip (takozvani Zynq Processing System) sa 1GB DDR3 memorije. Taj SoC predstavlja ujedinjenje velikog broja povezanih sistema koji međusobno rade i komuniciraju uz pomoć interne AXI4 magistrale. Od interesa za naš projekat su sledeće komponente i njihove uloge:

Komponenta	Veza i uloga
Zynq Processing System	ARM procesor, DDR kontroler, FCLK od 100 MHz i IRQ F2P ulaz.
AXI DMA	MM2S čita ulaz iz DDR-a, S2MM vraća rezultat u DDR; simple mode.
AXI GPIO	Kanal za run prema RTL-u i all done prema procesoru.
AXI SmartConnect	Povezuje PS master interfejs sa DMA i GPIO AXI Lite registrima.
AXI Memory Interconnect	Povezuje DMA memory mapped mastere sa PS HP DDR portom.
xlconcat	Spaja MM2S i S2MM interrupt linije na PS IRQ F2P.
Processor System Reset	Generiše sinhronizovane reset signale za AXI periferije.
IP jezgro (fa_axis)	Izvšava enkoder sloj i prima signale iz GPIO i AXI Stream komponenti.

Naše IP jezgro je predstavljen kao uređaj koji je povezan na AXI4 magistrali i komunicira sa navedenim komponentama. Posao našeg drajvera jeste da pravilno podesi odgovarajući hardver kako bi se omogućio rad našeg IP jezgra. Naša aplikacija i drajver će se pokretati unutar Linux operativnog sistema koji se pokreće na ARM procesoru. Kao što nam je poznato iz predavanja predmeta "Embedded operativni sistemi", nije moguće direkno upravljati sa navedenim hardverom unutar korisničkog programa i neophodno je napisati drajver koji će da preuzme tu ulogu.

Drajver predstavlja program koji se kreće kao privilegovan kernel-space proces unutar Linux operativnog sistema i za razliku od običnog user-space programa ima direktan pristup sistemskoj memoriji i magistralama bez virtuelne memorije.

Uzimajući u obzir da naš drajver radi sa fizičkim uredjajima koji se nalaze van opsega aplikativnog ARM procesora, moraćemo ih upravljati tako što definišemo registre i bafere koji zive na određenim memorijskim adresama. Takve adrese ćemo zapisivati u "stablo uređaja" (eng. device tree) koji će sadržati sve informacije o tome gde se koji uređaj nalazi u memorijskom prostoru. Kod Linux operativnog sistema takav "device tree" je strukturni tip podataka koji se učitava zajedno sa kernelom da bi kernelu prosledio informacije o tome gde se koji uređaj nalazi, kakve su veze signala izmedju uredjaja, informativni zapisi o tome kakav je tip uredjaja, kako se naziva, ko mu je vlasnik (ako uređaj ovisi od recimo ispravnog SPI kontrolera) i koji drajver Linux moze da koristi kako bi upravljao navedenim uređajem. Bez ovih informacija, svi sistemi koji nemaju nikakav drugi način da predstavi hardver će imati značajno ograničen rad jer neće moći proslediti drajverima neophodne informacije za njihovo upravljanje.

Zato kako bi smo pravilno napisali stablo uređaja neophodno je definisati dimenzije i ograničenja hardvera tj. modela sa kojim radimo. Mi u algoritmu od interesa izvršavamo enkoder slojeve Whisper Base modela, koje za jedan krug obrađenog sadržaja od 30 sekundi se izvršava 6 puta, svaki izračunati krug se naziva "slojem". Svaki taj sloj podrazumeva ulazne matrice Q, K i V za koje su poznatih dimenzija 1500 x 512, kao i izlazna matrica W koja je istih dimenzija.

Ovako velike matrice ne mogu da stanu u BRAM koji je dostupan FPGA jezgru, potom se operacija množenja treba podeliti na odgovarajuće delove koji mogu da stanu u BRAM koji se izvršavaju u isto vreme kako se ti podaci šalju niz magistralu uz pomoć AXI Stream-a. Zbog ovog razloga mi primenjujemo metodu Multi-Head Attention-a koja deli operaciju množenja na 8 glava koje se izvršavaju jedna nakon druge, tako da samo dio matrice mora biti prisutan u memoriji tokom množenja. Ova metoda zahteva da podaci budu poredani na određeni način kako bi se moglo izvršiti množenje, koja za zauzvrat ima efekat da se vrednosti K i V matrica ponavljaju za svaku glavu i 32 reda iz Q matrice. Ovo značajno povećava količinu podataka koji se trebaju poslati niz AXI Stream. Račun i obrazloženje je sledeći:

1. Prvo delimo operaciju na 8 identičnih glava koje obrađuju po 32 elementa po iteraciji iz reda od 1500 elemanata (zaokružujemo na viši broj jer 1500 nije deljivo sa 32, višak redova se preskače u izvršavanju algoritma).

2. Svaki od tih elemenata se dodeljuju 64 kolone iz Q matrice čiji broj dobijamo tako što podelimo broj kolona (512) sa brojem glava (8).

3. Zatim svakom tom deliću matrice Q od 32 x 64 elemenata se dodeljuje 1500 x 64 elemenata iz matrica K i V.

Svaki elemenat koji se šalje niz matricu je 24 bit vrednost koja je upakovana u 32 bitnu reč koji se šalje jedan po jedan uz pomoć AXI Stream-a. Izlazni deo sadrži samo 1500 x 512 elemenata u 64 bitnim rečima.

Matematički izraženo dobijamo sledeću formulu za neophodnu RAM memoriju za smeštanje ulaznog i izlaznog stream-a:

$$8 \text{ glava} \times (ceil(\frac{1500 \text{ redova}}{32}) \times (64 \text{ kolona po glavi} \times 32 \text{ reda} + 2 \times 1500 \text{ redova} \times 64 \text{ kolona po glavi})) \times 32 \text{ bit reči} = 278.328 \, MiB$$

$$1500 \text{ redova} \times 512 \text{ redova} \times 64 \text{ bit reč} = 5.86 \ \ MiB$$

Što u potpunosti čini 284.188 MiB neophodnog RAMa za čuvanje streamova koji šaljemo ka IP jezgru.

3. Neophodni preduslovi za razvijanje drajvera u Linux okruženju

Kako bi mogao započeti razvoj drajvera u Linux okruženju neophodno je posvetiti vreme na uspostaljanje ispravnog Linux okruženja u kojem možemo da pokrenemo našu aplikaciju i razvijati drajvere.

U slučaju Zybo Z7-10 ploče to će biti Xilinx/AMD okruženje nazvano PetaLinux. PetaLinux sačinjava skupinu alata komandne linije koji se pokreću na Linuxu kako bi se razvilo Linux okruženje koje će pokretati našu finalnu aplikaciju uz pomoć RTL modela. Razlog korišćenja je to što nam automatizuje proces u kome razvijamo navedeno Linux okruženje: njegovo nameštanje, konfiguraciju, generaciju i kompilaciju softvera.

Proces započinjemo prateći korake iz 11. i 12. labaratorijske vežbe iz predmeta "Embedded operativni sistemi". Zarad sažimanja čitavog procesa, pratimo poglavlja 1 do 3.1 bez ikakvih promena, a u koraku 3.2 preskačemo dio gde menjamo fajl platform-top.h unutar direktorijuma project-spec/meta-user/recipes-bsp/u-boot/files. Ovo je iz razloga što navedeni proces sprečava da se naša ploča upali jer traži konekciju ka TFTP serveru kojeg nismo podesili u procesu instalacije PetaLinux-a. Naredne korake pratimo isto kao i u vežbi osim kada trebamo da napišemo naše "stablo uređaja". U fajlu project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi zapisujemo sledeći sadržaj:

```
#include <dt-bindings/gpio/gpio.h>
/include/ "system-conf.dtsi"
/ {
    reserved-memory {
        #address-cells = <1>;
        #size-cells = <1>;
        ranges;

        fa_dma_pool: fa-dma-pool {
            compatible = "shared-dma-pool";
            reusable;
            size = <0x14000000>; /* 320 MiB */
            alignment = <0x100000>;
        };
    };

    fa: fa-accelerator {
        compatible = "ee,fa-1.0";
        dmas = <&axi_dma_0 0>, <&axi_dma_0 1>;
        dma-names = "mm2s", "s2mm";
        run-gpios = <&axi_gpio_0 0 GPIO_ACTIVE_HIGH>;
        done-gpios = <&axi_gpio_0 1 GPIO_ACTIVE_HIGH>;
        memory-region = <&fa_dma_pool>;
    };
};
```

U njemu opisujemo uredjaj nazvan fa-accelerator, koji traži drajver koji ispunjava ime uredjaja "ee,fa-1.0". Koristi dve AXI DMA kanale 0 (MM2S) i 1 (S2MM). S2MM i MM2S su Xilinx-ova konvencija za imenovanje smerova u kojima podaci putuju:

1. Stream to memory-mapped, odnosno slanje podataka iz memorije ka uređaju/IP jezgru. Mi kasnije ovaj kanal nazivamo u drajver kodu TX i sadržaće naše QKV matrice u navedenom formatu iz prethodnog dela ovog rada.

2. Memory-mapped to stream, odnosno slanje podataka iz uređaja/IP jezgra ka memoriji. Mi kasnije ovaj kanal nazivamo u drajver kodu RX i preko njega primamo završnu W matricu izračunatog sloja.

Tom uređaju se takođe dodeljuju dva GPIO signala (kojih pristupamo preko AXI magistrale):

1. RUN je izlazni signal i njemu govorimo kada može naše IP jezgro da započne sa radom

2. DONE je ulazni signal kojim naše IP jezgro vraća informacije o tome je algoritam završio i da se mogu preuzeti podaci.

Zadnji dio od značaja je rezervisana memorija kojom govorimo Linuxu da oslobodi kontinualni blok od 320MB radne memorije u koje bi smo mogli da zadržavamo čitavu dužinu sadržaja iz oba streama. Veličina koja je odabrana je veća od zapravo neophodne memorije kako bi smo osigurali dovoljno prostora u slučaju da neka kernel struktura ili nasumična memorija zauzme malo više radne memorije od tačno navedenog zbira.

Svi ostali koraci iz vezbe se nastavljaju isto kao i kod vežbe do poglavlja 4. Mi tu unosimo još jedan korak kojim generišemo bit fajl za naše FPGA jezgro koje možemo da dinamički učitamo za vreme pokretanja aplikacije. Ovo je neophodno uraditi kako bi smo mogli kasnije lagano resetovati i/ili reprogramirati FPGA jezgro tokom ravoja Linux driver-a u slučaju da se ono zaglavi tokom rada, kako bi smo mogli brže da isprobavamo nove izmene koda. Proces je sledeći:

1. U direktorijumu gde smo stvorili naš PetaLinux projekat, moramo da upalimo Xilinx Vivado-ov TCL shell.

2. Nakon što smo pokrenuli TCL shell unosimo komandu:

```
write_cfgmem -force -format bin -interface smapx32 -disablebitswap -loadbit "up 0
             images/linux/system.bit" images/linux/byte_swapped.bin
```

Ovo generiše novi fajl images/linux/byte_swapped.bin koji možemo da kopiramo zajedno sa ostalim boot fajlovima na SD karticu koju budemo napravili u koraku 4.

Dalji postupak se razlikuje po izboru Linux distribucije koja se kreće na ploči, u našem slučaju, koristi se Linux distribucija nazvana ArchLinux ARM za ARMv7 mikroarhitekturu. Proces instalacije neće biti opisan, jer je dosta sličan instalaciji bilo koje druge linux distribucije iz .tar.gz fajla i proces se može pratiti iz skoro bilo koje druge Linux distribucije.

Nakon što budemo instalirali željenu Linux distribuciju na SD kartici, započinjemo proces pokretanja Linux-a na Zybo ploči time što pomeramo jumper J5 na poziciju označenu sa SD. Zatim ubacujemo našu spremljenu SD karticu u Zybo ploču i ubadamo je microUSB kablom u naš radni računar.

Ako smo sve uspešno namestili, naša Zybo ploča će se pojaviti kao USB serijski uredjaj koji možemo pristupiti time što podesimo baud rate na 115200 i posmatramo prosec palenja. Nakon par minuta pokretanja, trebao bi nam se pojaviti getty login shell u koji se možemo ulogovati da započnemo rad na instalaciju neophodnih alata za razvoj drajvera i aplikacije.

- Čim se ulogujemo u Linux okruženje, moramo ploču povezati na internet kako bi se mogli preuzeti neophodni razvojni alati. To radimo tako što spajamo RJ45 LAN kabl između naše Zybo ploče i našeg rutera sa LAN portovima.
- Kada isprobamo da je internet ispravan, prvi korak je da uradimo pun upgrade Linux sistema. U slučaju Arch Linux-a to se izvršava komandom `pacman -Syu` kao root koriskik.
- Nakon toga neophodno je instalirati sledeće alate: gcc, g++, make, autoconf, automake, cmake, pkg-config, git i (potencijalno) text editor ako nameravamo da menjamo kod na samoj ploči. Ostale promene koje nisu navedene su urađene radi lakšeg korišćenja ploče i nisu neophodne za sam rad programa.

Ovo obuhvata sve neophodne izmene i podešavanja neophodne na ploči kako bi smo mogli započeti razvojem Linux driver-a na Zybo ploči.

4. Drajver i aplikcije

Unutar git repo-a projekta, ulazimo u direktorijum `esl/driver`. Iz tog foldera vidimo sledeće direktorijume: app, bitstream, driver. Driver sadrži naš drajver koji pokrećemo na ploči. Bitstream sadrži bit fajl koji smo generisali kao zadnji korak pred stavljanja Linux-a na SD karticu. A app sadrži dva foldera: whisper.cpp i test.

4.1. Drajver

Kod drajvera se sadrži iz dva fajla: fa_axis.c i fa_axis.h. Naziva se fa_axis po imenu top fajla RTL modela.

U fa_axis.h-u se nalaze opisane dimenzije i ograničenja našeg akceleratora u #define preprocesorskim direktivama, gde ih nasledjuju proračunate dimenzije matrica i njenih delova. Nastavlja se opisom strukture fa_info kojom mi korisnickim programima možemo da kažemo (iz kernel prostora) šta naš drajver podržava u veličinama TX i RX bafera zajedno sa količinom reči od kojih se one sadrže. FA_MMAP_TX i FA_MMPA_RX su konstante koje koristimo za offset parametar u mmap() kernel pozivu kako bi smo odredili da li korisnik zeli ulazni ili izlazni bafer. Zadnje tri linije opisuju makroe koje koristimo za ioctl naredbu kako bi smo mogli upitati drajver za veličinu matrice preko FA_IOC_GET_INFO i FA_IOC_RUN kako bi smo poslali signal za pokretanje Flash Attention algoritma na IP jezgru. Neophodno je imati ovaj odvojeni header fajl kako bi korisničke aplikacije mogle koristiti taj isti header fajl da bi ubacili neophodne makroe za pravilan rad.

Funkcionisanje drajvera prvenstveno ovisi od pravilno definisanog stabla uređaja kojeg smo obradili u poglavlju 3 ovog dokuemnta. On nam imenima dodeljuje sledeće uredjaje:

Ime	Uloga
run	GPIO izlaz kojim pokrećemo IP jezgro
done	GPIO ulaz kojim nam IP jezgro govori da je završen algoritam
mm2s	AXI4 Stream kojim šaljemo Q, K i V matrice ka uređaju
s2mm	AXI4 Stream kojim primamo izlaznu W matricu

Kao i neophodnu zaključanu kontinualnu regiju memorije od 320MiB za ovaj uređjaj specificno u kojoj ćemo da smestimo bafer za stream koji izgrađujemo kako bi naš RTL model mogao pravilo da sračuna vrednosti matrice. Razlog zašto je ova regija kontinualna jeste zbog toga što goli DMA kontroleri očekuju da memorija bude bez prekida ili premeštanja i generalno ne uzimaju fragmentaciju memorije ili virtuelne adrese u obzir.

Prva stvar koju drajver radi pri učitavanju je to što pokreće funkciju fa_probe čija namera u Linux-u je da generiše sve interne strukture i resurse za rad drajvera. Generalno drajveri koji očekuju više instanci jednog uređaja (ili sličnih ako isti drajver podržava više vrsta uređaja) koriste probe funkcije kao način da pretraže sistem da bi ih prepoznali sve instance i korigovati resurse za njih. Uzimajući u obzir da se ne očekuju veći broj instanciranja, napravili smo drajver na takav način da alocira neophodne strukture za samo jednu instancu uređaja fa_axis.

To instanciranje podrazumeva:

Podatak/Varijabla	Uloga
fa	Struktura u kojoj držimo sve ostale navedene resurse
fa->lock	Mutex kojim blokiramo dupli pristum DMA regiji unutar drajvera
fa->rx_done	Oznaka za ispunu RX DMA prenosa
fa->tx_done	Oznaka za ispunu TX DMA prenosa
fa->run	Objekat koji predstavlja GPIO izlaz za pokretanje IP jezgra
fa->done	Objekat koji predstavlja GPIO ulaz za pregledanje da li RTL model tvrdi da je završio
fa->mm2s	Objekat koji predstavlja DMA kanal kojim šaljemo sastavljen Q, K i V bafer.
fa->s2mm	Objekat kojim predstavljamo DMA kanal kojim primamo izlaznu W matricu.
fa->tx_cpu, tx_dma	Pokazivač ka baferu namenjen za MM2S DMA kanal.
fa->rx_cpu, rx_dma	Pokazivač ka baferu namenjen za S2MM DMA kanal.
fa->misc	Handle koji predstavlja MISC tip karakter uredjaja kojom sa aplikacije komuniciraju sa drajverom.

Koristimo of_reserved_mem_device_init kao način da kernelu kažemo da smo mi vlasnici one rezervisane memorije i da je možemo koristiti. Tokom stvaranja objekata za GPIO i DMA kanale pozivamo se na imena iz stabla uređaja u kome smo naveli ista imena. Na takav način Linux pravilo dodeljuje pokazivače i zna o kojem se uređaju radi.

Karakter uređaj je način kojim mi kao razvijač drajvera se predstavljamo Linux-u i aplikacijama koje nameravaju da koriste naš uređaj. Koristimo strukturu fa_fops kako bi smo rekli Linux-u koje funkcije da pozove u slučaju da neki proces pokuša da pristupi karakternom uređaju koji smo mi nazvali "fa_axis" i time smestili u lokaciji /dev/fa_axis gde će biti vidljiv korisniku i njegovim/njenim aplikacijama. Linux tretira sve uređaje kao fajlove pa zato funkcije koje moramo implementirati kako bi naš drajver radio ispravno prate konvencije poziva fa fajl uređaje. Funkcije koje su implementirane su:

Funkcija	Uloga
fa_open (tj. open)	Funkcija kojom korisnički program pokušava da otvori navedeni uređaj/fajl.
fa_mmap (tj. mmap)	Fukncija kojom korisnički program ubacuje navedeni fajl u svoj memorijski prostor.
fa_ioctl (tj. ioctl)	Funkcija kojom se obrađuju ne-standardne operacije. Obično se koristi za upite, nameštanja ili slanje signala.

U fa_open-u mi samo prosleđujemo "fajl" pokazivač koji predstavlja naš uređaj tako da ga možemo koristiti kasnije kada korisnik/proces pokuša da uradi mmap poziv.

fa_ioctl je poziv kojim regulišemo stanje uređaja tj. pokretanje (FA_IOC_RUN) i vraćamo informacije korisničkom programu o dimenzijama bafera koje DMA/IP očekuje (FA_IOC_GET_INFO). FA_IOC_RUN nakon provere mutex-a pokreće funkciju fa_run koja implementira prenos navedeni DMA podataka i čitavo održavanje i proveravanje stanja tokom tog čitavog procesa izvršavanja, u detalje kasnije ulazimo.

fa_mmap koristi prebačeni pokazivač iz fa_opena kako bi memorijski mapovao ili TX ili RX bafer u zavisnosti od vrednosti koje stavimo u offset tog mmap poziva. Tip memorijskog prenosa u mmap pozivu koji koristimo je WRITE_COMBINE jer on nam omogućava da naši memorijski zapisi budu keširani i time brži za CPU.

fa_run je ubedljivo najbitnija funkcija u čitavom drajveru jer ona prati čitav proces prenosa podataka izmedju bafera i DMA kao i regulisanja stanja uređaja. Redosledom se dešavaju operacije:

1. Proverava da li taj uređaj trenutno vrši neku operaciju i prijavljuje grešku u slučaju da več kreće algoritam.

2. Zatim podešavaju RX i TX prenosi (tj. širina njihovih reči - 64 i 32 bit).

3. Priprema se RX prenos, podešava se callback funkcija koja će da setuje oznaku za završetak RX DMA prenosa i pokreće se asinhron RX DMA prenos.

4. Sinhronizuju se sve memorijske operacije preko mb() naredbe i pokreće se jezgro tako što se toggluje pin "run" preko GPIO signala.

5. U for petlji se šalju TX bafer u komadima od 4MB preko DMA TX kanala. Razlog je to što Xilinx-ovi DMA pozivi u default Linux kernel konfiguraciji su ograničeni da imaju maksimalne prenose do 8MB. Isto važi kao i za RX prenos: pokreće se asinhrono, podešavaju se callback funkcije i tajmer koji će da prati koliko dugo svaki poziv trajti i da li će da premaši određeno vreme. Ako predugo bude trajao, prijavljuje se greška i prekida se izvšavanje algoritma.

6. Nakon završetka for petlje, čeka se završetak RX DMA prenosa, ako on ne stigne u odgovarajuće vreme, izbacuje se greška.

7. Na kraju se proverava stanje "done" GPIO pin-a i prijavljuje se greška da je završio DMA prenos ali da done signal nije podignut.

4.1.1. Kompilacija drajvera

Da bi se pokrenuo drajver neophodno ga je prvo kompajlirati. Unutar kloniranog projektnog git repo-a, neophodno je ući u folder driver/driver u kome se pokreće make komanda da bi se kompajlirao drajver pod uslovom da su svi header fajlovi pravilno instalirani sa vežbe.

4.2. Test aplikacija

Da bi smo isprobali pravilnost drajvera koji smo razvili za našu Zybo ploču, neophodno je bilo napraviti aplikaciju koja će simulirati račun jednog sloja enkodera Whisper AI modela. Program će prosto da uzme primer ulaza i izlaza enkoder sloja koji smo sačuvali kao testne podatke tokom rada na RTL delu projekta. U projektnom repo-u postoje dva fajla unutar rtl/software foldera:

Fajl	Uloga
fa_real_qkv.bin	Fajl koji sadrži (unapred pravilno formatirane za FPGA) Q, K i V aktivacije koje šaljemo na IP blok.
fa_real_golden.bin	Rezultate koje očekujemo kao rezultat dobrog proračuna tih Q, K i V vrednosti koje smo validirali eksterno, tj. očekivani W rezultat proračuna tog sloja.

U folderu driver/app/test se nalazi ta testna aplikacija. Ukratko opisano, ona radi sledeće korake:

1. Otvara /dev/fa_axis karakterni uređaj koji je naš drajver stvorio.

2. Proverava dimenzije matrice da li se poklapaju sa onim šta IP očekuje.

3. Upituje sistem koja je veličina memorijske stranice kako bi mogao da koristi offset parametar da bi rekao drajveru da li se radi o TX ili RX baferu. Zatim koristi mmap pozive da memorijski mapira te adrese u svoj adresni prostor.

4. Učitava u potpunosti oba gore navedena testna podatka.

5. Igrađuje redosled matrica neophodan da se pošalje uređaju kako bi Multi-Head attention algoritam ispravno radio. Opisan je proces detaljnije u 3. poglavlju ovog dokumenta.

6. Koristi monotonične tajmere da bi precizno izmerio vreme prije i posle pokretanja algoritma preko ioctl poziva, da bi mogao da prijavi vreme izvršavanja.

7. Poredi dobijen RX rezultat sa učitanim golden podatkom i prijavljuje grešku u slučaju da neka vrednost ne jednači, u suprotnom prijavljuje uspeh.

4.2.1. Kompilacija i pokretanje testne aplikacije

Za pokretanje aplikacije, neophodno je kompajlirati isto kao što smo kompajlirali i drajvere. Prelaskom u direktorijum driver/app/test, pokrećemo make komandu.

Pokretanje algoritma se izvršava skriptom run_fa.sh koja se nalazi u folderu driver. Ta skripta traži root dozvole jer prvo reprogramira FPGA čip, pa zatim učita drajvere, pa pokreće testnu aplikaciju.

4.3. Whisper.cpp

Whisper.cpp je aplikacija komandne linije kojom korisnici uglavnom pokreću Whisper AI model. Veoma je popularna jer podržava veliki izbor hardvera za ubrzavanje izvršavanja AI modela. Neki od uređaja koje podržava su:

- AI akceleratori (NPU – Neural processing unit)

- Grafičke kartice (GPU – Graphical processing unit) direkno preko CUDA kernela ili preko opštih grafičkih programskih interfejsa kao što su Vulkan ili OpenCL

- Raznovrsne CPU-specifične optimizacije koje uzimaju u obzir mikroarhitekturu datog procesora na kome kreću kako bi ubrzao izvršavanje matematičih operacija programa

Cilj uvezivanja našeg Whisper Base akceleratora za Whisper.cpp projekat jeste da demonstriramo moguću praktičnu implementaciju ovog projekta tako što se uvezujemo za popularnu implementaciju koja se koristi širom sveta. Naravno, ovo neće biti potpuna implementacija akceleratora koja pokriva sve moguće operacije, jer naš RTL model je opredeljen na samo izvršavanje enkoder slojeva Base modela specifično koji ima zadate dimenzije (1500 x 512 po 6 slojeva). Naš kod koji uvezuje akcelerator za ovaj projekat će specifično raditi samo u tim slučajevima.

Implementacija je napravljena tako što se uvezujemo na postojeću CPU implementaciju (takozvani "CPU backend") GGML biblioteke. GGML biblioteka je interna biblioteka projekta Whisper.cpp-a koja implementira Whisper AI algoritam za razne uređjaje, dok je Whisper.cpp samo interfejs kroz koji se taj algoritam predstavlja na korisnički lep način. Uvezivanje je postignuto time što u postojećoj funkciji ggml_compute_forward_flash_attn_ext od CPU GGML backend-a pozovemo funkciju ggml_accel_try_fa_axis gde pokrećemo naš kod. Ta funkcija vraća rezultat true ako je uspela da obradi sloj, a false ako nije tako da CPU backend može da preuzme ulogu u tom slučaju kako ne bi došao do kraha algoritma.

Implemencija te funkcije i čitavog backend-a za naš Flash Attention akcelerator je opisan u novom fajlu accel-fa-axis.h. Koraci koji GGML kod izvršavaju su skoro pa identični kao kod testne aplikacije, samo sa par izmena:

1. Pre nego što se algoritam počne izvršavati mi proveravamo da li trenutni sloj koji treba da se obradi je odgovarajućih dimenzija ili formata. Ako nije, mi taj sloj odbijamo i prepuštamo CPU backend-u da odradi posao umesto akceleratora. Tu ulogu izvršava funkcija accel_shape_supported i provere parametara softcap i max_bias, koje nismo implementirali kao parametre koje možemo proslediti akceleratoru.

2. Moramo da zaključamo operaciju na jedno jezgro jer je podrazumevano da se algoritam kreće na više niti kako bi se ubrzalo njegovo izvršavanje. Ovim sprečavamo dupli pristup podacima, a svakako nemamo više od jednog akceleratora tako da nam je potrebna samo jedna nit kako bi smo pokrenuli algoritam.

3. Koristimo funkciju accel_build_stream kako bi smo izgradili TX stream veoma slično kao i kod testnog koda, samo sa primarnom izmenom da se aktivacije trebaju prevesti sa internog float16 ili float32 formata na naš traženi Q6.18 format za ulazne vrednosti i Q14.34 za izlazne vrednosti matrice. Funkcije koje su zaslužne za prevođenje tih vrednosti su accel_f32_to_word i accel_word_to_f32. Funkcije accel_elem i accel_emit_row su samo prevodioci sa internog GGML strukture aktivacija na naš oblik koji akcelerator traži. Funkcija accel_elem takođe uzima u obzir skaliranje ulaznih Q vrednosti preko parametra q_scale, jer naš Flash Attention RTL algoritam to ne radi automatski i treba se implementirati kao dodatni korak.

4. Interno vođenje stanja, tipa file descriptora i ostalih resursa su namerno stavljeni u "procurenu" memoriju kako ne bi smo ih trebali ponovo inicijalizovati za svaki sloj. Ovo ne šteti radu našeg programa jer ta memorija svakako je potrebna čitavom dužinom izvršavanja algoritma, a čim program završi je oslobođena od strane Linux-a koji vodi računa koji proces zadržava koje memorijske resurse. Ispravan destuktor je pravilno implementiran kako bi Linux mogao da ispravno oslobodi resurse. Iz ovog razloga je implemetiran kao C++ klasa.

5. Tokom izvršavanja algoritma se prijavljuju vremena obrade slojeva na sličan način i greške ukoliko se pojave.

6. Na kraju su naše izlazne vrednosti W koje smo dobili iz akceleratora prevedene iz Q14.34 natrag u floating point brojeve koje postavljamo na adekvatne pozicije u GGML matricama. To postižemo tako što čitamo dobijene rezultate iz akceleratora i uz pomoć tri for petlje smeštamo natrag na pozicije gde trebaju da stoje kako bi GGML biblioteka ispravno dalje obrađivala.

Ako su naši fa_axis drajver i RTL model sve uspešno odradili, funkcija ggml_accel_try_fa_axis vraća true, i to se koristi kao signal u CPU implementaciji da je naš akcelerator odradio taj sloj i da CPU ne mora da ga obrađuje.

4.3.1. Kompilacija i pokretanje Whisper.cpp projekta

Da bi smo kompajlirali Whisper.cpp, fakultetski repo mora biti kloniran sa –recurse-submodules, jer neophodna grana whisper.cpp projekta se nalazi na GitHub repozitorijumu a ne na repozitorijumu projekta zbog njene veličine.

Prilikom pokretanje aplikacije, neophodno je kompajlirati Whisper.cpp isto kao što smo kompajlirali i drajvere. Prelaskom u direktorijum driver/app/whisper.cpp, pratimo sledeće komande (podrazumevajući da su svi predhodni koraci instaliranja neophodnih paketa u Linux okruženju iz poglavlja 3 ispoštovani):

```
mkdir build

cmake -B build -DCMAKE_BUILD_TYPE=Release \

      -DGGML_NATIVE=OFF -DGGML_CPU_ARM_ARCH=armv7-a \

       -DCMAKE_C_FLAGS="-mfpu=neon-fp16 -mfloat-abi=hard" \

       -DCMAKE_CXX_FLAGS="-mfpu=neon-fp16 -mfloat-abi=hard"

cmake --build build -j2
```

Proces kompilacije whisper.cpp projekta na Zybo ploči je izuzetno spor (traje par sati) zbog slabog procesora koji se sadrži samo od dva ARM Cortex A9 jezgra koji kreću maksimalnom brzinom od 666 MHz. Build flagovi su ovde neophodni jer koriguju optimizacije GCC kompajlera da uzme u obzir nativne floating point operacije prisutne na samom CPU jezgru. Inače podrazumevano je da ih nema, pa učini pokretanje ostalih koraka Whisper modela izuzetno sporim jer se računaju preko floating point emulacije u softveru. Ovi flagovi značajno ubrzavaju rad Whisper.cpp-a, čak i do 20 puta.

Nakon uspešne kompilacije projekta, prebacujemo se natrag u direkorijum "driver" unutar projektnog foldera gde koristimo istu skriptu kao kod test programa, run_fa.sh. Skripta će da primetiti da je whisper.cpp kompajliran i prisutan, time će početi da sam preuzima težine Whisper Base modela sa Huggingface-ove web stranice gde stoje u punom kvalitetu. Nakon što preuzme težine Whisper Base modela, automatski će ih konvertovati u niži kvalitet (Q5_0) putem kvantizacije kako bi smo dobili model koji je značajno brži za izvršavati na slabim CPU jezgrima Zybo ploče. Nakon što se završi taj proces, automatski će pokrenuti whisper-cli komandu koja će izvršiti analizu podatka normal.wav koji se nalazi u direktorijumu "data" unutar projektnog foldera. Taj podatak je testni snimak audio govora snimljen čisto da se istestira ispravnost izvršavanja modela. Nakon izvršavanja algoritma, dobijamo izlazni tekst sa pratećom oznakom vremena koji nam opisuje rezultat detekcije govora tog audio sadržaja.

5. Performanse i rezultati

Prilikom izvšavanja testne aplikacije i whisper.cpp programa, utvrdili smo da je potpuno ispravno funkcionisanje algoritma time što smo kod testnog programa uporedili rezultate sa golden podatkom i videli da su sve vrednosti jednake, a kod whisper.cpp-a smo dobili ispravne rezultate transkripcije govora kod fajlova normal.wav i whisper.wav. Izvršavanje jednog sloja enkodera kod testnog programa

se obavlja za 3.4 sekunde, što je također isti rezultat koji dobijemo kada koristimo našu verziju whisper.cpp programa.

Performanse na zalost ne stižu realno vreme potrebno za uživo transkripciju govora, a to je iz više razloga:

- Ubrzali smo samo enkoder slojeve Whisper Base modela, a ostali koraci kao što su FFN (Feed-Forward Network), dekoder slojevi i finalna projekcija se još dalje izvršavaju na CPU jezgrima Zybo ploče koje smo ustanovili da su relativno malih performansi.

- Značajan dio vremena gubimo u prevođenju težina relativno na njihov račun. Primer: Jedan enkoder sloj se prevede u 1.98 s, dok njegov proračun traje samo 3.4s, to jest čak 37% ukupnog vremena izvršavanja. Površnom istragom saznajemo da je promet podataka ugrađene DDR3 memorije ograničenje.

- Čak i da prevedemo ostatak mreže na RTL model, velika je verovatnoča da bi smo bili ispod vremena neophodnog za uživo prevod govornog sadržaja jer kada gledamo idealne rezultate, bez ostalih gubitaka kao što su CPU operacije i konverzija vrednosti iz jednog u drugi format, primećujemo da preostalo vreme za račun je veoma mal. Pun krug izvršavanja inferense Whisper Base modela obrađuje sadržaj ekvivaletan 30 sekundi audio snimka. Prevođenje audio fajla u tokene traje 7.1 sekundi, gde svaki sloj na RTL-u se izvrši u 3.4 sekunde. Pomnožimo 3.4 sekunde sa 6 slojeva i zbir toga svega ostavlja nam samo 2.5 sekunde dosupno u prozoru od 30 sekundi! Ostatak operacija kao što su FFN i dekoder slojevi bi trebalo nekako smestiti unutar tog bužeta od 2.5 sekundi što ispada veoma neizvodljivo uzimacijući u obzir da samo jedan enkoder sloj traje 3.4 sekunde.

Uzimajući u obzir ove nedostatke, još dalje se jako dobro poklapamo sa projektovanim vremenom iz ESL i RTL delovima projekta, što znači da je naša implementacija (makar u tehničkom vidu) upsešna.

5. Zaključak

Uspešno je implementiran Linux kernel drajver koji koristi Flash Attention IP blok da bi ubrzao rad obrade enkoder slojeva Whisper Base modela. Drajver uspešno uspostavlja sve neophodne preduslove da IP blok krene obrađivati podatke, koje verifikacijom kroz testni program i Whisper.cpp smo utvrdili da su ispravni. Jedini nedostatak je u performansama prevođenja govora u realnom vremenu, ali to su ograničenja koja svakako nisu praktična za rešavanje kada se uzme u obzir preostalo vreme prerade u idealnom slučaju bez gubitaka tokom procesorske obrade.